

PROJET DATA LAKE

Rapport Data Lake Financier

Architecture, pipeline, API Gateway et resultats verifies

Equipe

Artemiy Smogunov, Nathan Smadja-Tubiana et Patrice Ignongui

Theme

Finance uniquement

Module

Data Lakes & Data Integration - EFREI 2025-2026

Professeur

Yvann Vincent

Date

7 juillet 2026

Validation technique effectuee sur Docker, Airflow, FastAPI, MinIO, Elasticsearch, PostgreSQL et Redis.

Sommaire

- Introduction et objectifs
- Presentation des donnees
- Architecture generale
- Pipeline Airflow
- Zones Raw, Staging et Curated
- API Gateway
- Niveau avance et optimisations
- Resultats de verification
- Procedure de lancement
- Conclusion

Introduction et objectifs

Ce projet met en place un data lake de bout en bout sur le theme finance. Le choix du domaine financier est coherent avec les series temporelles du sujet: il permet de manipuler des donnees OHLCV, de calculer des indicateurs techniques, puis de detecter des comportements anormaux sur les marches. L objectif technique est de couvrir la chaine complete: ingestion depuis deux sources, stockage brut, transformations vers des zones structurees, exposition par API Gateway, orchestration automatique et benchmark avance entre un endpoint standard et un endpoint optimise.

Perimetre retenu - Le theme est volontairement limite a la finance: aucune autre thematique n est melangee au projet.

Presentation des donnees

Le data lake combine deux sources de donnees financieres, conformement aux consignes du projet.

Source

Role

Format

Exemples

Dataset fichier local

Backdata historique et
reproductible

CSV

finance_dataset.csv,

principalement AAPL

API Yahoo Finance

via yfinance

Actualisation quotidienne et
batches a la demande

Donnees

tabulaires OHLCV

AAPL, MSFT, GOOGL,

indices ^GSPC, ^DJI,

^IXIC

Les champs exploites sont les prix Open, High, Low, Close, Adjusted Close, Volume, le ticker et la date.

Ces champs sont adaptes aux calculs de moyennes mobiles, RSI, MACD, bandes de Bollinger, volatilité et rendement journalier.

Architecture generale

L architecture suit une logique medaillon Raw, Staging, Curated. Les composants sont conteneurises avec Docker Compose afin de garantir un demarrage reproductible.

Couche

Technologie

Justification

Raw

MinIO + Elasticsearch

MinIO conserve les fichiers bruts; Elasticsearch permet une recherche rapide dans les evenements OHLCV.

Staging

PostgreSQL

Donnees nettoyees et structurees avec indicateurs techniques.

Curated

PostgreSQL

Donnees enrichies par detection d anomalies et signaux de trading.

Orchestration

Apache Airflow

DAG planifie du lundi au vendredi a 6h UTC, avec reprise et logs.

Couche

Technologie

Justification

API Gateway

FastAPI

Endpoints /raw, /staging, /curated, /health, /stats, /ingest,

/ingest_fast.

Cache

Redis

Cache optionnel pour accélérer /ingest_fast.

Pipeline Airflow

Le DAG principal se nomme `financial_data_lake_pipeline`. Il orchestre l'ensemble du flux de données financier.

1. start: point d'entrée du DAG.

2. ingest_file et ingest_api: ingestion parallèle depuis le dataset fichier et depuis yfinance.

3. transform_staging: consolidation en PostgreSQL avec calcul des indicateurs techniques.

4. transform_curated: enrichissement par Isolation Forest, scoring d'anomalies et classification.

5. log_summary: résumé de pipeline dans les logs Airflow.

6. end: fin contrôlée du workflow.

La planification est configurée sur `0 6 * * 1-5`, donc une exécution les jours ouvrables à 6h UTC. Le DAG utilise LocalExecutor, des retries et XCom pour propager les résultats entre tâches.

Zones du Data Lake

Raw

La zone Raw conserve les données les plus proches possible de la source. Les fichiers sont stockés dans MinIO et les lignes OHLCV sont indexées dans Elasticsearch.

- Bucket `raw-financial-data`: fichiers CSV par ticker et par date.
- Bucket `raw-api-data`: traces de données issues de l'API.
- Index `raw_financial_events`: recherche par ticker, date, source et valeurs OHLCV.

Staging

La zone Staging normalise les données en table `staging_ohlc`. Les données y deviennent directement exploitables par API et par traitements analytiques.

- Calculs: SMA 20/50, EMA 12/26, RSI 14, MACD, bandes de Bollinger.
- Mesures: `daily_return` et `volatility_20`.
- Gestion des doublons par `upsert ticker/date`.

Curated

La zone Curated contient les données finales. Elle ajoute des scores d'anomalie, des flags et des catégories utiles pour l'analyse financière.

- Modele: Isolation Forest sur rendement, volatilité, volume z-score et RSI.
- Sorties: anomaly_score, is_anomaly, anomaly_type et signal buy/sell/hold.
- Objectif: identifier des chocs, pics de volume et comportements atypiques.

API Gateway

FastAPI expose une interface unique pour interroger et alimenter le data lake sans accéder directement aux bases ou aux buckets.

Endpoint

Usage

GET /health

Etat des dependances: PostgreSQL, MinIO, Elasticsearch, Redis.

GET /stats

Volumes par couche et metriques de remplissage.

GET /raw

Recherche dans la zone Raw via Elasticsearch.

GET /staging

Lecture des donnees OHLCV enrichies par indicateurs.

GET /curated

Lecture des donnees finales avec anomalies et signaux.

POST /ingest

Pipeline synchrone standard, mesure de reference.

POST /ingest_fast

Pipeline optimise pour le niveau avance.

Niveau avance et optimisations

Le niveau avance demande un endpoint /ingest, un endpoint /ingest_fast et une amelioration d au moins 30% sur les memes tailles de batch. Le projet couvre ces points.

Optimisation

Implementation

Effet attendu

Parallelisation

ThreadPoolExecutor avec 8 workers

Telechargement yfinance et uploads MinIO plus rapides.

Vectorisation

NumPy et pandas rolling

Calculs techniques reduits sans boucles Python inutiles.

Bulk Elasticsearch

helpers.bulk

Moins d allers-retours reseau pour l indexation.

Batch PostgreSQL

execute_values

Insertion/upsert plus efficace que des requetes

unitaires.

Cache Redis

TTL de 5 minutes si use_cache=true

Evite les telechargements repetes en test ou

demo.

Resultats de verification

Les controles suivants ont ete executes apres relance de Docker Desktop et des services Docker

Compose.

Controle

Resultat verifie

Services Docker

PostgreSQL, MinIO, Elasticsearch, Redis, Airflow et API lances; services

critiques healthy.

API /health

overall=ok; PostgreSQL, MinIO, Elasticsearch et Redis OK.

Airflow /health

metadatabase healthy et scheduler healthy.

Import DAG

No data found, donc aucune erreur d import.

Tests unitaires

6 tests executes, 6 OK.

DAG manuel

codex_finalcheck_20260707T1734Z en success, 7 taches success.

Volumes apres relance

Zone

Volume verifie

Raw MinIO

533 objets au total: 421 raw-financial-data et 112 raw-api-data.

Raw Elasticsearch

5930 documents indexes.

Staging PostgreSQL

5899 lignes dans staging_ohlcv.

Curated PostgreSQL

4891 lignes dans curated_analysis.

Logs ingestion

28 lignes dans ingestion_logs.

Benchmark avance

Batch

/ingest

/ingest_fast

Gain

Statut

1 ticker

4160.22 ms

1863.95 ms

55.20%

success, 0 erreur

100 tickers

70631.54 ms

23612.12 ms

66.57%

success, 0 erreur

Formule: gain = (temps standard - temps optimise) / temps standard x 100. Les deux mesures dépassent le seuil demande de 30%.

Procédure de lancement

- 1. Ouvrir Docker Desktop et attendre que le moteur Docker soit disponible.**
- 2. Depuis le dossier du projet, lancer: docker compose up -d --build.**
- 3. Verifier les services avec: docker compose ps.**
- 4. Verifier l'API: <http://localhost:8000/health> et la documentation: <http://localhost:8000/docs>.**
- 5. Ouvrir Airflow: <http://localhost:8080> avec admin/admin, puis déclencher le DAG**

financial_data_lake_pipeline si besoin.

- 6. Ouvrir MinIO: <http://localhost:9001> avec minioadmin/minioadmin.**

Accès - Airflow utilise admin/admin. MinIO utilise minioadmin/minioadmin. Ces identifiants sont adaptés à un environnement local de projet.

Conclusion

Le projet répond aux exigences du sujet: deux sources de données, architecture Raw/Staging/Curated, pipeline Airflow planifié, API Gateway complète, endpoints avancés et benchmark documenté. La relance du 7 juillet 2026 confirme que les services, tests, DAG et volumes fonctionnent correctement.

Conclusion de validation - Le projet est opérationnel et le niveau avancé est valide par des

gains superieurs a 30% sur batch 1 et batch 100.